Simple ML Algorithms and General Principles [1]

Shaobo Li

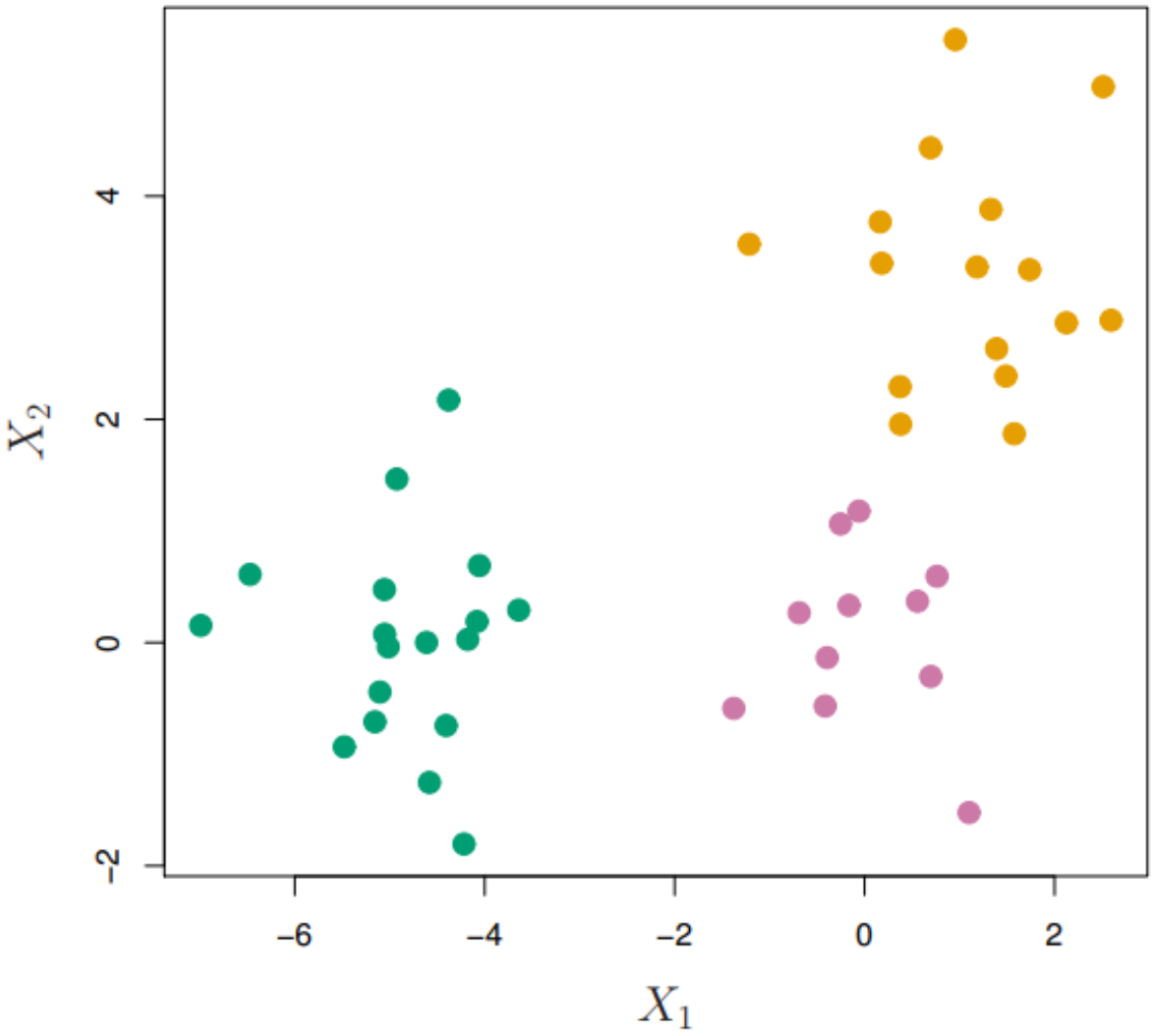
University of Kansas

[1] Partially based on Hastie, et al. (2009) ESL, and James, et al. (2013) ISLR

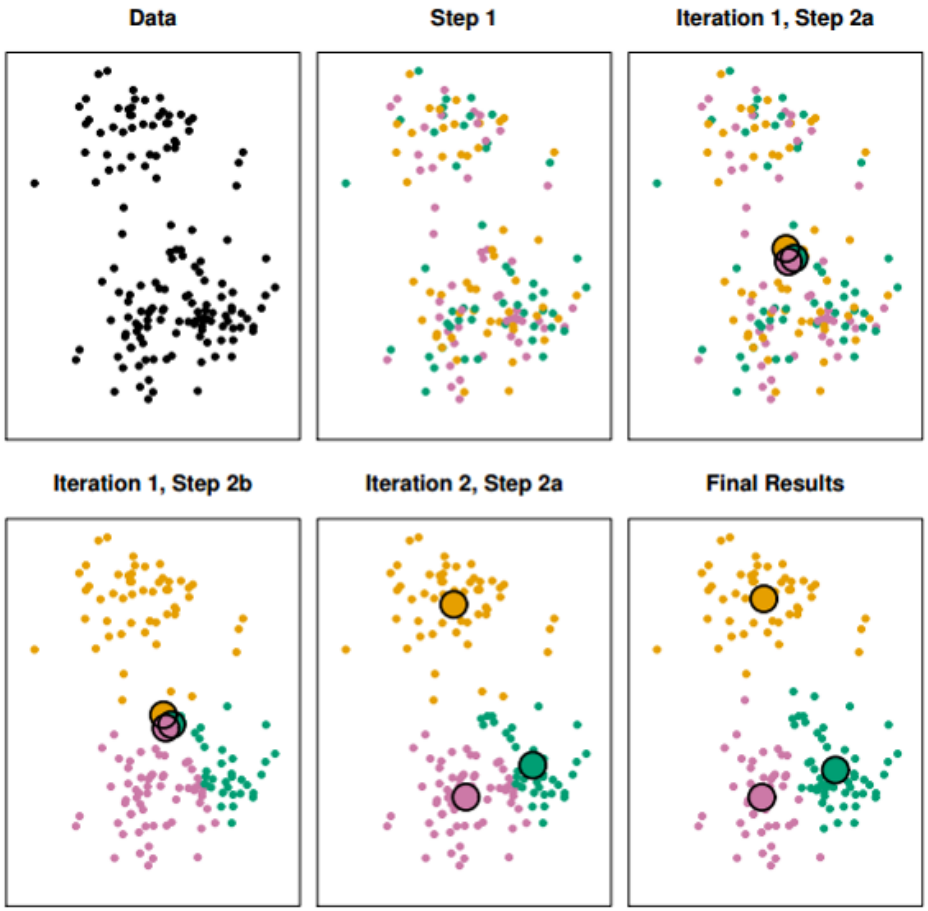
Unsupervised learning

- Unlabeled
- The goal is to detect structure of the input data
- Popular ML algorithms:
  - Clustering: k-means, hierarchical, and model based
  - Dimension reduction: principle component analysis (PCA)

K-means Clustering



How it works



- Here is a very good animation to illustrate k-means clustering algorithm. [link]

How it works

1. Randomly find k data points (observations) as the initial centers
2. For each data point, find the closest center and label it (e.g., using different colors). Now you have k clusters
3. Re-calculate the centers of current clusters
4. Repeat step 2 and 3 until the centers do not change

Can you implement the algorithm with R?

Why does it work?

It is all about variance/distance/error! For a random variable X with mean of μ,

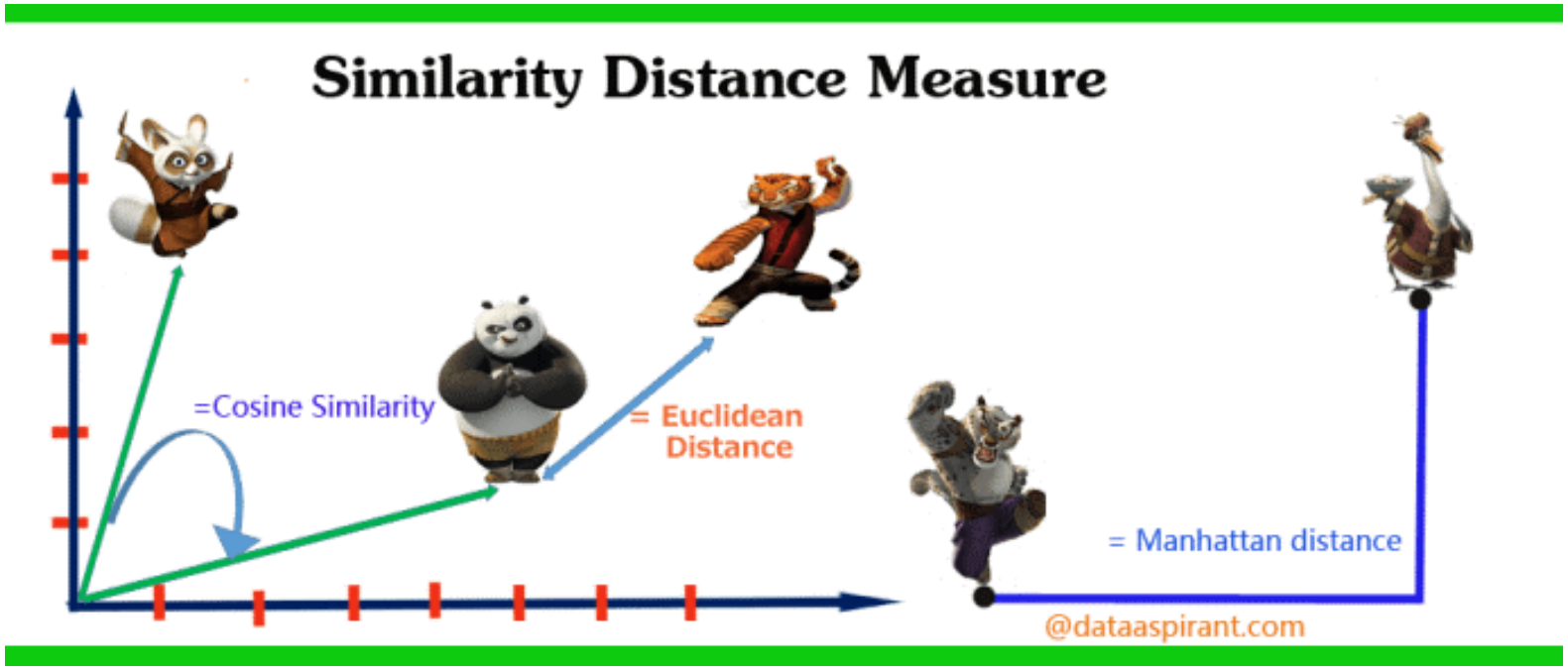
$$Var(X) = E(X - \mu)^2$$

Suppose that $X_1, ..., X_n$ are observed values of X, then

$$\widehat{Var}(X) = \frac{1}{n-1} \sum_{i=1}^{n} (X_i - \bar{X})^2$$

- Minimizing variance within clusters
- Maximizing variance between clusters
- How about total variance?

Distance-based Similarity Measures



Practice

Download the data file here, and try R function `dist()`.

X1	X2	X3
16.6	1.2	6.0
17.9	0.7	6.9
15.9	1.0	4.2
15.4	0.4	4.5
17.0	1.1	5.1
16.3	1.0	5.5
14.3	1.5	1.4
15.7	0.1	5.0
15.4	0.9	1.4
16.4	1.8	1.7
17.2	0.7	5.3
15.3	0.6	3.9
15.9	0.9	4.7

Supervised Learning

- Labeled data
- The goal is to predict or explain certain outcome
- Type of problem:
  - Regression: outcome is continuous
  - Classification: outcome is categorical
- Popular ML algorithms:
  - Least square, nearest neighbor, CART, gradient boosting, neural network, deep learning

Supervised Learning – Least Squares

- Linear regression model

$$Y = \beta_0 + \beta_1 X_1 + \ldots + \beta_p X_p + \epsilon$$

- The estimated model is

$$\hat{Y} = \hat{f}(\mathbf{x}) = \hat{\boldsymbol{\beta}}^T \mathbf{x}$$

- Solve $\hat{\beta}$ using least square

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} (y_i - \boldsymbol{\beta}^T \mathbf{x}_i)^2$$

- Exercise: derive the analytical solution using matrix format.

Supervised Learning – Nearest Neighbor

- No model assumption
- Nonparametric method

$$\hat{Y} = \hat{f}(\mathbf{x}) = \frac{1}{k} \sum_{\mathbf{x}_i \in N_k(\mathbf{x})} y_i$$

  where $N_k(\mathbf{x})$ is the neighborhood of $\mathbf{x}$ defined by the k closest points $\mathbf{x}_i$.
- k determines the flexibility of the model (should larger k or smaller k results in more flexible model?)
- How to define the neighbor? (How to find closest points to $\mathbf{x}_i$?)
  - Similarity measures

An example[2]

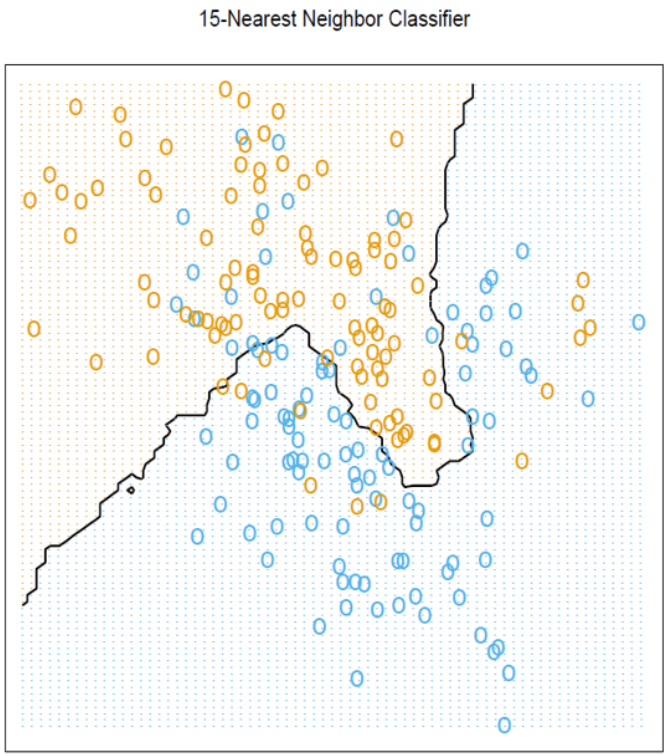


FIGURE 2.2. *The same classification example in two dimensions as in Figure 2.1. The classes are coded as a binary variable* (BLUE = 0, ORANGE = 1) *and then fit by 15-nearest-neighbor averaging as in (2.8). The predicted class is hence chosen by majority vote amongst the 15-nearest neighbors.*

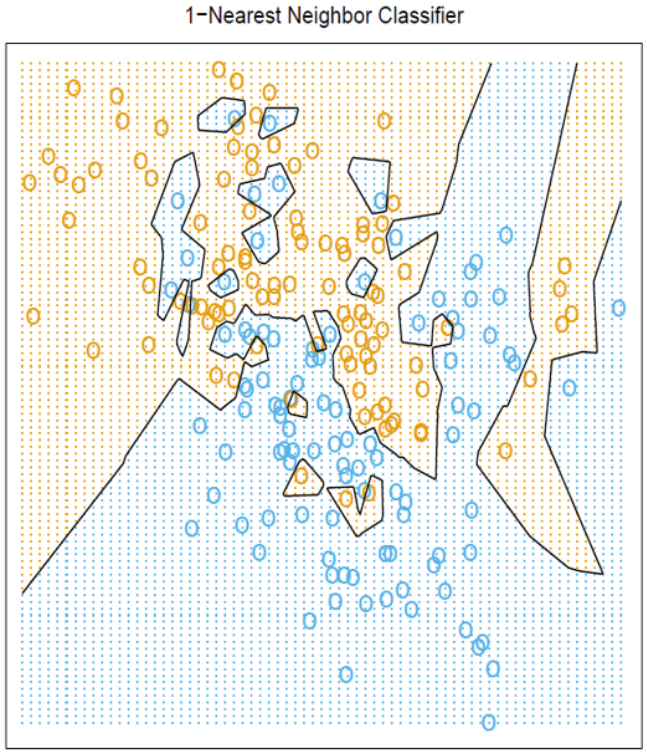


FIGURE 2.3. *The same classification example in two dimensions as in Figure 2.1. The classes are coded as a binary variable* (BLUE = 0, ORANGE = 1), *and then predicted by 1-nearest-neighbor classification.*

[2] ESL pg.15-16

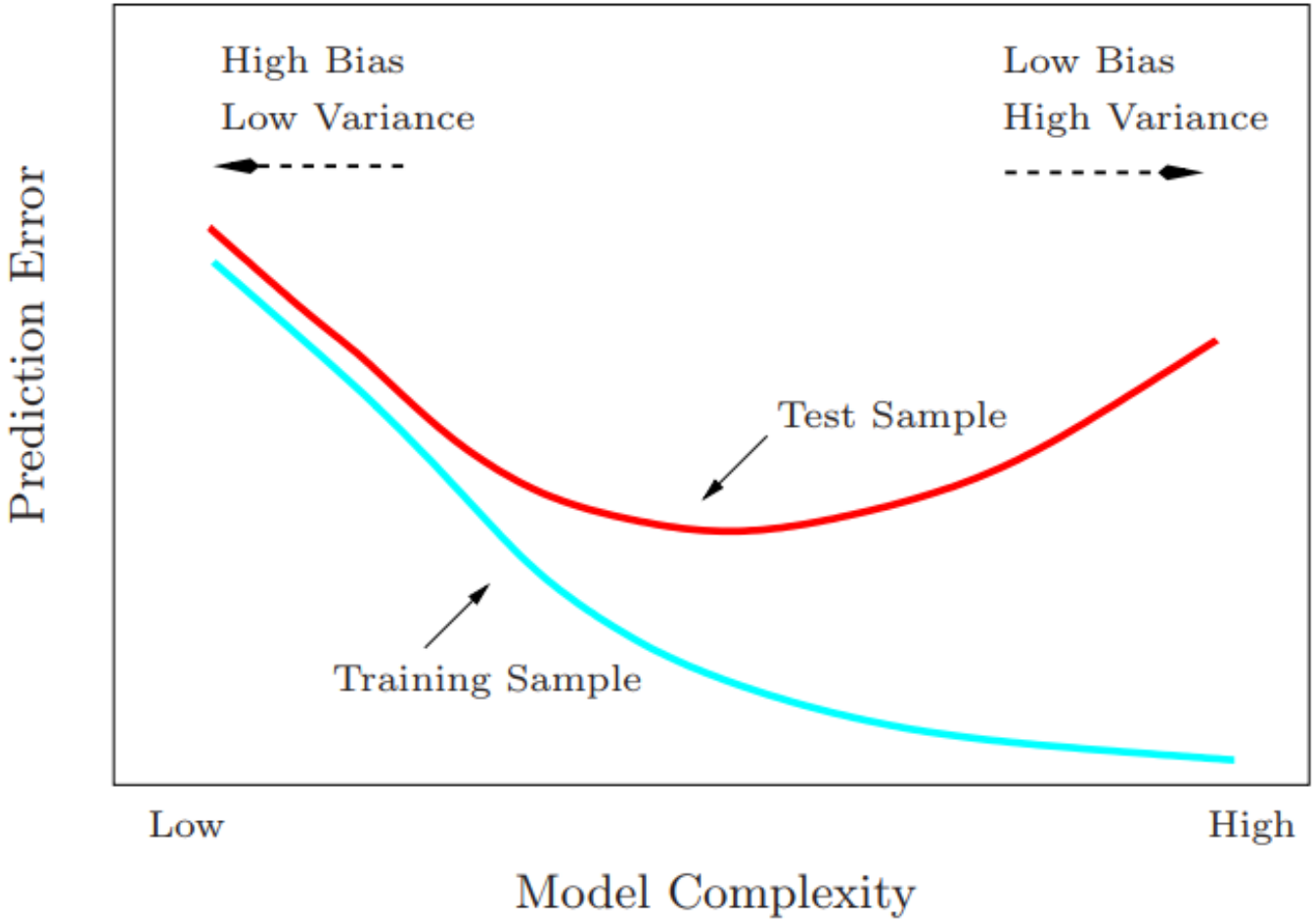
Model Assessment

How do we know if the estimated model $\hat{f}(x)$ is **useful**?

- We never know the true $f(x)$
- Split sample to training and testing sets
- Train the model (learning algorithm) based on training sample by minimizing *training error*
- Apply the estimated model on the testing sample to calculate the *prediction (testing) error*
- We care more about testing error rather than training error

HOW MUCH ACCURACY DID YOU GET ?
I GOT 99%
AND WHAT WAS THE PROPORTION OF YOUR TEST SET ?
I TESTED WITH THE SAME TRAINING SET
imgflip.com

Prediction Error



Prediction Error

One of the most common metrics is the *mean squared error*

- Denote training set $Tr = \{x_i, y_i\}_1^N$, and testing set $Te = \{x_i, y_i\}_1^M$

$$\text{MSE}_{Tr} = \frac{1}{N} \sum_{i \in Tr} (y_i - \hat{f}(x_i))^2$$

$$\text{MSE}_{Te} = \frac{1}{M} \sum_{i \in Te} (y_i - \hat{f}(x_i))^2$$

- Training error, MSE_{Tr}, may be biased due to overfitting
- In this course, we denote *MSE* as training error, and *MSPE* as testing error

Classification Problems

- Response variable is **qualitative**
- Train a classifier $\hat{C}(x)$ that can label any new input data x
- It usually involves certain decision rule
- Prediction (testing) error: Misclassification rate (MR)

$$MR_{Te} = \frac{1}{M} \sum_{i \in Te} \mathbb{I}[y_i \neq \hat{C}(x_i)]$$

K-Fold Cross-validation

- Instead of doing training vs. testing once, we do it K times

Folder 1	Folder 2	Folder 3	Folder 4	Folder 5
Test	Train	Train	Train	Train
Train	Test	Train	Train	Train
Train	Train	Test	Train	Train
Train	Train	Train	Test	Train
Train	Train	Train	Train	Test

- Use 2,3,4,5 as training and 1 as testing
- Use 1,3,4,5 as training and 2 as testing
- Keep doing this loop...
- Average 5 testing errors, that is CV score

Leave-one-out Cross-validation

- By the name, it requires to repeat training-testing procedure n times
- However, for least square linear model, there is a short cut that makes LOOCV the same that of a single model fit

$$CV_n = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - \hat{y}_i}{1 - h_i}\right)^2$$

  where h_i is the diagonal element of "hat" matrix.
- In general, the estimates from LOOCV are highly correlated hence their average can have high variance
- In practice, $K = 5$ or 10 is recommended

Bias-Variance Tradeoff

- This is a very important tradeoff that governs the choice of statistical learning methods.
- **Bias**: how far the estimated model $\hat{f}(x)$ is to the true model $f(x)$.
  - Unbiased estimate is defined as: $\mathbb{E}\hat{f}(x) = f(x)$
  - Usually, we calculate the squared bias: $(\mathbb{E}\hat{f}(x) - f(x))^2$
- **Variance**: the variation of estimated model $\hat{f}(x)$ based on different training set.

Bias-Variance Tradeoff

Source: link

Bias-Variance Tradeoff

- Suppose the data arise from a model $Y = f(x) + \epsilon$, with $\mathbb{E}(\epsilon) = 0$ and $Var(\epsilon) = \sigma^2$.
- Suppose $\hat{f}(x)$ is trained based on some training data, and let (x_0, y_0) be a test observation from the same population.
- The *expected prediction error* can be decomposed to:

$$\mathbb{E}[y_0 - \hat{f}(x_0)]^2 = \sigma^2 + Bias^2(\hat{f}(x_0)) + Var(\hat{f}(x_0))$$

(Exercise: Show it.)

- Typically as the **flexibility** of $\hat{f}$ increases, its variance increases, and its bias decreases.